Smart Page Replacement Algorithm

An improved strategy compared against FIFO and LRU

Team 12

2520030074 K. Bhargavi

2520030603 M. Siri Chandana

2520030300 G. Nandini

Guide

Dr. Bheekya

Operating Systems | Memory Management

Why page replacement matters

- Physical memory is small, programs ask for more than fits.
- When a page is missing and every frame is full, the OS must evict one.
- A fault means a disk read, which is roughly 100,000 times slower than a RAM access.
- So the choice of victim decides how fast the whole system feels.

Goal

Fewest page faults for the same reference string and the same number of frames.

The two standard baselines

FIFO

Evicts the page that has been in memory the longest.

Simple and cheap to run.

Ignores usage, so a hot page can be thrown out just because it is old. Can show Belady's anomaly: more frames, more faults.

LRU

Evicts the page not used for the longest time.

Follows locality well and is the usual benchmark.

Only remembers the last touch. One long scan of cold pages can push out pages that are used again and again.

The idea: remember often, forget slowly

1 **Count uses**
Every page keeps a small counter that goes up on each access.

2 **Age the counters**
After every 4 x frames accesses, all counters are halved so old popularity fades.

3 **Evict the weakest**
Victim is the page with the lowest counter. A tie goes to the least recently used.

How it runs on each page request

1. **Page is already in memory**
   Count it as a hit and add 1 to its counter.

2. **Page is missing**
   Count it as a page fault.

3. **Memory is full**
   Remove the page with the lowest counter. If tied, remove the one used longest ago.

4. **Load the new page**
   Give it a starting counter of 1.

Why it works

New pages start at 1, so a one-time scan is the first thing evicted.

Counters are halved regularly, so old favourites do not stay forever.

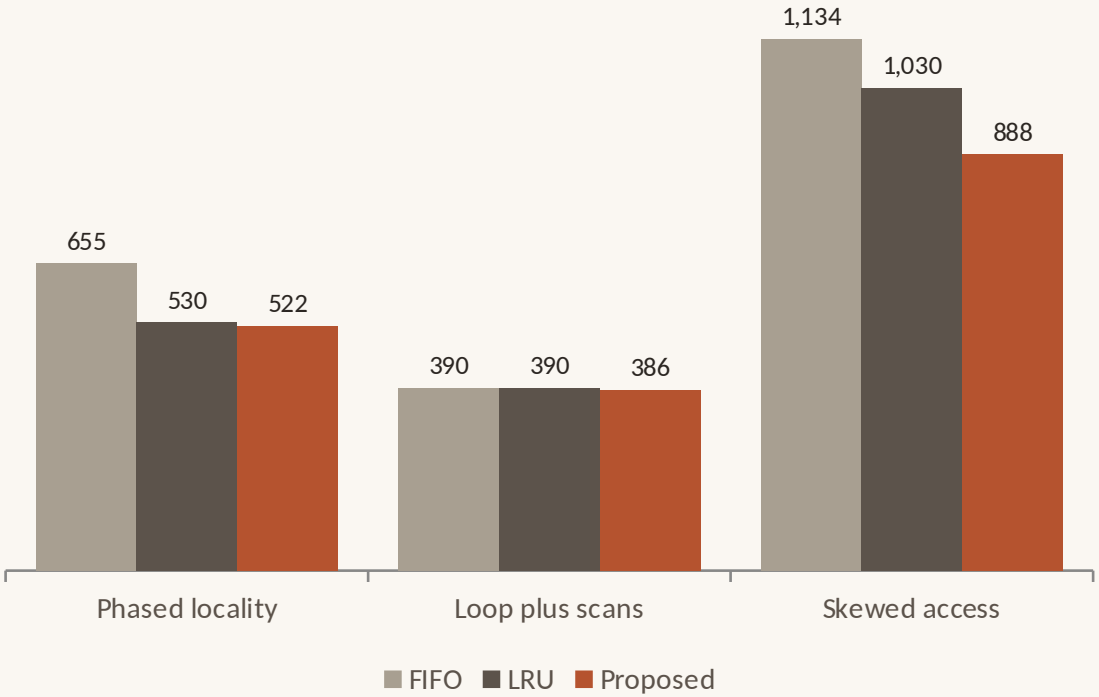
Cost is just one small counter per frame.

How we compared them

Same reference string and same frame count for every algorithm. We counted page faults and also ran the optimal algorithm (OPT) as a lower bound.

Workload	Pattern	Requests	Frames
Phased locality	Small working set that shifts 8 times, 10% stray pages	2000	8
Loop plus scans	A hot loop of 5 pages interrupted by long one-time scans	1080	6
Skewed access	A few pages are requested far more than the rest	2000	10

Results: page faults (lower is better)



vs LRU

Phased: 1.5% fewer faults

Scans: 1.0% fewer

Skewed: 13.8% fewer

vs FIFO

20% to 22% fewer on phased and skewed workloads.

What we learned

- Combining frequency with recency beat both FIFO and LRU on every workload we tried.
- The gain is large when some pages are clearly hotter than others, and small when access is mostly sequential.
- OPT is still far ahead (for example 620 faults on the skewed run), so there is room to improve.
- Limits: the halving interval was fixed at 4 x frames, and these are synthetic strings, not real program traces.
- Next step: tune the interval automatically and test on real traces.